

Level-Distinguishability Operational Mechanisms

Derivation Note B2.09 — CKS Theory Series, Paper 2 Derivation

Wenxin Li

Independent Researcher

ORCID: 0009-0004-8065-3235

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Date: May 12, 2026

This work derives from and formalizes the instinct/reasoning separation pattern introduced in “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), the second paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026).

Abstract

Paper 2 commits to three architectural levels—cell, aspect, and Self—each with distinct scope. B2.07 formalizes the level-distinct scope property; B2.08 formalizes level-membership as substrate-resident. This note, B2.09, formalizes the operational mechanisms by which the three levels are kept distinct in deployment. Level-distinguishability is not a property that holds automatically from the existence of the three-level structure; it requires specific architectural mechanisms that enforce level distinctions operationally. Six such mechanisms are identified and formalized: substrate-resident type declarations per A2.46; scope-validation rules per A2.04; cross-level operation provenance per A2.40; level-specific governance per A2.47; level-specific testing per A5.01–A5.16; and inspection semantics per A2.01. These mechanisms are mutually reinforcing—none suffices alone, and deployment correctness depends on multiple mechanisms operating together. The note articulates why operational distinguishability is architecturally necessary, how it differs from nominal level distinction in conventional AI architectures, the inherited Paper 1 commitments it depends on, and its operational implications for deployment.

1. Why Level-Distinguishability Operational Mechanisms Require Formalization

B2.07 established that the three CKS levels—cell, aspect, and Self—have distinct scope: the cell handles a specific informational task; the aspect groups cells for a purpose-defined mode of engagement; the Self holds multiple aspects and their collective intelligence as one whole. B2.08 established that level-membership—the record of which artifacts participate at which level—is substrate-resident, not implicit in runtime configuration. Together these two notes establish what the

levels are and how membership is recorded. What neither establishes is the operational question: how are those levels kept distinct as deployment evolves over time?

The question is not academic. In any sufficiently long-lived deployment, operations accumulate, rules are added and revised, new artifacts are introduced, governance responsibilities shift, and the architecture is tested against unanticipated conditions. Under those conditions, level distinctions that are nominal—expressed only in naming conventions, documentation, or team agreements—erode. A governance operation configured at cell-level scope gets applied inadvertently to an aspect-level artifact. A rule authored for one level gets invoked at another. An operation intended to reorganize a Self’s aspect structure silently collapses into cell-level operations. These are not hypothetical pathologies; they are the natural consequence of absence of operational enforcement.

CKS addresses this through operational mechanisms: architectural structures that enforce level distinction as a runtime property, not merely a design-time intention. These mechanisms make the level distinctions first established in B1.02 actually maintainable in deployment.

This is the ninth Phase B2 note and the third of four notes decomposing B1.02. B2.07 formalized the level-distinct scope property. B2.08 formalized level-membership as substrate-resident. B2.09 (this note) formalizes the operational mechanisms that maintain level-distinct scope in deployment. B2.10 will formalize level-instantiation patterns, closing the B1.02 decomposition. Subsequent Phase B2 notes move to B1.03 cell-level decomposition.

The strategic prior-art posture of this note is precise: it formalizes that level-distinguishability requires multiple explicit architectural mechanisms, that those mechanisms must be substrate-resident rather than implicit, and that the specific mechanisms identified here—type declarations, scope-validation, provenance, level-specific governance, level-specific testing, and inspection semantics—jointly constitute the operational architecture that makes level-distinct scope per B2.07 actually maintainable.

2. The Six Operational Mechanisms

Level-distinguishability in CKS deployment operates through six mechanisms. Each contributes a distinct dimension of enforcement; together they constitute the operational architecture for maintaining level-distinct scope.

Mechanism 1: Substrate-Resident Type Declarations (A2.46)

Each artifact in the CKS substrate is declared as a cell, an aspect, or a Self. The type declaration is part of the authoritative substrate content per A2.46 Category 4—not a property inferred from runtime context, not derivable from an artifact’s name, and not configurable through LLM reasoning. The declaration specifies which level the artifact participates at. It includes provenance per A2.40: when the artifact was declared as type X, by whom, under what rule, and under what authority. Because the type declaration is substrate-resident and provenance-tracked, it is inspectable and auditable. No artifact exists in the substrate without a declared level-type; no level-type can be changed without producing a

new provenance record.

Mechanism 2: Scope-Validation Rules (A2.04)

Scope-validation rules are authored per A2.04 as orchestration rules that specify valid operation targets by level. A cell-level operation—one whose subject is a cell-level artifact—cannot be applied to an aspect-level artifact without an explicit scope-crossing rule authorizing that cross-level operation. Similarly, an aspect-level operation cannot be applied at Self scope without explicit authorization. Scope-validation rules are themselves substrate-resident, authored by humans, and auditable. They operationalize the level-distinct scope property: level-distinct scope is not a passive description of the architecture but an actively enforced constraint at the rule layer. Where a deployment needs to permit cross-level operations—as B1.19 explicitly supports—the scope-validation rules explicitly authorize those crossings, making them governed rather than accidental.

Mechanism 3: Cross-Level Operation Provenance (A2.40)

When operations span levels—per cross-level access per B1.19 or per vertical evolution per B1.14—the six provenance metadata fields per A2.40 explicitly record the cross-level character of those operations. The provenance record includes that the operation was cross-level, the source level, the target level, the authorization rule, the actor, and the timestamp. This means audit can identify all cross-level operations in a deployment’s history and verify that each was explicitly authorized as such. Cross-level operation provenance does not prohibit cross-level operations; it ensures that cross-level operations are distinguished from level-confined operations in the record. The absence of such distinction in a deployment record is itself an auditable finding.

Mechanism 4: Level-Specific Governance (A2.47)

The authority distribution mechanism per A2.47 supports level-specific authority assignment. Some humans hold cell-level governance authority—they are authorized to make decisions, author rules, and configure operations at cell scope. Others hold aspect-level governance authority. Others hold Self-level governance authority. This is not a rigid hierarchy preventing cross-level authority; it is a default assignment structure that makes governance operations level-aware. When a governance decision is made at one level, the record identifies both the governance actor and the level at which their authority applies. Level-specific governance prevents a class of level-confusion errors that occur when governance operations are level-agnostic: a human with cell-level authority inadvertently configuring aspect-level behavior, or a human with aspect-level authority applying rules at Self scope without explicit elevation.

Mechanism 5: Level-Specific Testing (A5.01–A5.16)

The operational tests per A5.01–A5.16 apply at each level distinguishably. When a test is run at cell level, it verifies cell-level properties—that the cell’s orchestration rules, conflict-handling, provenance, and substrate commitments hold. When the same test logic runs at aspect level, it verifies aspect-level properties—that the aspect’s coordination of its constituent cells, its purpose-defined arrangement, and its scope commitments hold. When run at Self level, it verifies Self-level properties—that the Self’s

integration of aspects, its unified governance, and its wholeness commitments hold. Level-specific testing is distinct from level-agnostic testing: a test that does not distinguish levels cannot verify that level-distinct scope is maintained, only that some aggregate behavior holds. The level-specific application of the operational test set is what provides deployment-time verification that level-distinguishability is operationally maintained.

Mechanism 6: Inspection Semantics (A2.01)

The inspect right per A2.01—humans can inspect substrate content at any time—is given level-aware semantics in a three-level deployment. When a human inspects a cell, the cell-level content is visible: the cell’s substrates, rules, DNA layer, action layer, provenance records, and membership records. When a human inspects an aspect, the aspect-level content is visible: the cells that are members, the coordination rules governing their arrangement, the aspect’s purpose definition, and the aspect’s provenance—without collapsing into cell-level content display. Inspecting an aspect does not automatically display all cell-level content of all member cells; the level boundary is maintained in the inspection response. When a human inspects a Self, the Self-level content is visible: the aspects that are members, the Self’s unified governance structure, the Self’s topology, and the Self’s provenance. Level-aware inspection enables humans to ask level-distinguished queries: “show me all cells in this aspect,” “show me all aspects in this Self,” “show me the cross-level operations in this Self’s history.” These queries return level-distinguished answers, making the three-level structure usable for human governance and audit rather than merely recorded in substrate.

3. What Makes Level-Distinguishability Operational Mechanisms Architecturally Distinctive

The six mechanisms described above jointly constitute something that conventional AI architectures typically lack: operational enforcement of level distinctions rather than nominal designation.

In conventional multi-agent or orchestrated AI architectures, components are often described as occupying different levels—an “orchestrator” operates at a different level than a “worker agent,” a “meta-agent” operates at a different level than a “sub-agent.” These level designations are real in that they reflect design intent and inform system behavior. But the mechanisms enforcing those designations are typically soft: naming conventions, documentation, team agreements, configuration files that could be changed. When a deployment evolves—new agents added, rules revised, integrations extended—the soft mechanisms erode. An agent initially designated as a worker agent gets invoked at orchestrator scope; a meta-agent’s rules get applied at sub-agent scope; governance configured for one level silently applies at another. There is no architectural mechanism preventing or recording this erosion.

CKS’s operational distinguishability mechanisms are hard in the relevant sense: substrate-resident type declarations cannot be changed without producing a new provenance record; scope-validation rules prevent operations from crossing levels without explicit authorization; cross-level operation

provenance records every instance of cross-level access; level-specific governance assigns authority at the level of architectural record rather than informal agreement. The level distinctions are operationally enforced, not nominally claimed.

This distinction matters specifically because level confusion compounds. An operation that inadvertently crosses a level boundary often produces downstream effects at multiple levels; those effects produce further operations; the level-confused state propagates through the deployment. Operational enforcement of level distinguishability prevents this propagation at the source by making level-inappropriate operations either blocked by scope-validation rules or explicitly authorized and recorded as cross-level. In either case, the level boundary is maintained in the operational record.

The second distinctive feature is that the six mechanisms are mutually reinforcing. Type declarations alone do not prevent level confusion if scope-validation rules do not enforce type-declared level boundaries. Scope-validation rules alone do not create an auditable record of cross-level access if cross-level operation provenance is absent. Cross-level operation provenance alone does not prevent inadvertent cross-level governance if level-specific authority distribution is not operational. Level-specific governance alone does not verify that level-distinct properties hold if testing is not level-specific. Level-specific testing alone does not make the three-level structure usable for human governance if inspection semantics are not level-aware. Each mechanism plugs a gap that the others leave open. The deployment correctness of level-distinguishability depends on multiple mechanisms operating together, not on any single mechanism in isolation.

4. The Cognitive Analog as Conceptual Scaffold

Paper 2's framing draws on the analogy between CKS's three architectural levels and the scoped cognitive operations observable in human cognition. Specific perceptual recognition operates at a granular, stimulus-proximate scope—recognizing a face, parsing a sentence, identifying a sound. Mode of engagement operates at an intermediate scope—the competitive sports mode, the calm study mode, each grouping specific cognitive operations under a purpose-defined orientation. Integrated personhood operates at the scope of the whole person—the unified sense of self that holds multiple modes as facets of one individual without collapsing them into each other.

Human cognition keeps these scopes distinct through mechanisms that are not merely nominal: perceptual recognition operations do not automatically trigger restructuring of one's integrated identity; mode-of-engagement transitions do not erase the granular recognitions they are built upon; the wholeness of personhood is not reducible to the sum of its mode-level constituents. The cognitive system maintains scope distinctions operationally through the structure of cognitive processing, not merely through labeling.

The CKS architectural analog is direct: the three levels are kept distinct not through naming but through operational mechanisms that enforce scope boundaries, record cross-scope operations, maintain level-specific authority, and support level-aware inspection. The cognitive analog provides conceptual

scaffolding for why this architecture is coherent—it mirrors a structural distinction that human cognition is empirically known to maintain. The architectural substance, however, is the six operational mechanisms and the inherited Paper 1 commitments they depend on; the cognitive analog is scaffolding, not argument.

5. Inherited Paper 1 Commitments

Level-distinguishability operational mechanisms depend on and extend several Paper 1 commitments, all inherited without modification.

A1.02 — Substrate-cell boundary.

The substrate-cell boundary per A1.02 is enforced at each level through type declarations. A cell is bounded by the substrate-cell boundary at cell scope; an aspect is bounded by the substrate-cell boundary at aspect scope; a Self is bounded by the substrate-cell boundary at Self scope. Level-distinguishability preserves A1.02 at every level, not only at cell scope.

A2.46 — Authoritative substrate content.

Type declarations are Category 4 authoritative content per A2.46—content that defines the identity and properties of substrate artifacts. The scope-validation rules that enforce level boundaries are also authoritative substrate content. Distinguishability mechanisms are not runtime configuration; they are substrate-resident authoritative content, readable, inspectable, and governed.

A2.04 — Rule authoring.

Scope-validation rules are authored per A2.04 as orchestration rules by humans with appropriate authority. The rule-authoring process produces provenance-tracked, substrate-resident rules that specify valid operations by level. The scope-validation mechanism is entirely within the rule-authoring architecture Paper 1 establishes.

A2.40 — Six provenance metadata fields.

Cross-level operation provenance uses the six metadata fields per A2.40 as its record structure: what the operation was, who performed it, under what authority, when, what substrate content was affected, and under what rule. Provenance of cross-level operations is not a new record structure but an application of the existing six-field provenance commitment to the cross-level dimension of operations.

A2.01 — Inspect right.

Level-aware inspection semantics extend the inspect right per A2.01 to the three-level structure. Humans can inspect at any time; in a three-level deployment, inspection is level-aware, returning level-distinguished content rather than collapsed flat content. The inspect right is not weakened by the three-level structure; it is given level-appropriate semantics.

A2.47 — Authority distribution.

Level-specific governance extends the authority distribution mechanism per A2.47 to three levels. Authority can be assigned at cell, aspect, or Self scope; the record of authority includes the level scope of that authority. Authority distribution is substrate-resident and provenance-tracked, as A2.47 specifies.

A5.01–A5.16 — Operational tests.

The operational test set per A5.01–A5.16 applies at each level, with test logic running level-specifically rather than level-agnostically. The test set is not modified to support three-level deployment; it is applied level-specifically as deployment verification.

6. Operational Implications

Level-distinguishability operational mechanisms have several direct deployment implications.

During artifact birth per B1.06, new artifacts receive type declarations. The type declaration is part of the birth record: the artifact is born as a cell, an aspect, or a Self, with the type included in the birth provenance. No artifact enters the substrate without a declared level-type.

Scope-validation rules are configured per deployment requirements. Some deployments may require strict scope enforcement: no cross-level operations without explicit authorization. Others may permit more permissive scope crossing for particular operation classes while requiring strict enforcement elsewhere. The configuration of scope-validation rules is a deployment design decision made by humans with rule-authoring authority per A2.04; the decision is itself substrate-resident and provenance-tracked.

Vertical evolution per B1.14—structural reorganization that modifies the level relationships among artifacts—produces type declaration modifications. A cell that is reorganized into an aspect through vertical evolution receives a new type declaration, with provenance recording the reorganization event, the authority under which it was made, and the prior type declaration. Type declaration history is part of the artifact’s provenance chain.

Cross-level access per B1.19 has explicit recording mechanisms through cross-level operation provenance per A2.40. Deployments that permit cross-level access—for example, a Self accessing a cell directly when purpose requires, as Paper 2 explicitly supports—accumulate a record of those access events. Deployment audit can verify that cross-level access occurred only within authorized scope-crossing rules, and can trace cross-level access events to the purposes they served.

Level-specific testing is scheduled per deployment verification needs. Deployments that modify cell-level properties run cell-level tests; deployments that modify aspect-level structure run aspect-level tests; deployments that reorganize Self-level topology run Self-level tests. The scheduling decision is governed by humans with testing authority; the test results are substrate-resident records.

Inspection tools support level-aware queries. A human governance actor can ask “show me all cells in this aspect” and receive a cell-list scoped to that aspect’s membership. A human auditor can ask “show me all cross-level operations in this Self’s history” and receive a provenance-filtered list of cross-level events. Level-aware queries make the three-level structure usable for routine governance and audit, not merely recordable in substrate.

Deployment audit verifies level-distinguishability is operationally maintained by checking: that all artifacts have valid type declarations with complete provenance; that all operations are either level-confined or explicitly authorized as cross-level by scope-validation rules; that cross-level operations have complete provenance records; that governance operations are performed by actors with appropriate level-scope authority; that level-specific tests have been run and passed; and that inspection responses maintain level boundaries.

7. Limits

The scope of level-distinguishability operational mechanisms is specific, and several apparent implications are explicitly excluded.

Level-distinguishability mechanisms do **not** prevent cross-level operations. Cross-level access per B1.19 is explicitly supported by the architecture. Distinguishability mechanisms ensure that cross-level operations are authorized, recorded, and distinguishable in the provenance record—not that they are prohibited. The architecture permits cross-level access where purpose requires; it requires that such access be governed.

Level-distinguishability does **not** eliminate relational role-membership per B2.08. The same artifact can hold membership records at multiple levels—a cell participating in multiple aspects, for example. Distinguishability mechanisms govern operations on artifacts by their declared level-type, not by their membership records. The membership records per B2.08 and the type declarations per Mechanism 1 are distinct substrate content serving distinct purposes.

Level-distinguishability does **not** prescribe specific implementation mechanisms for type declarations, scope-validation rule structure, or provenance record format beyond the commitments of A2.40, A2.04, and A2.46 that are already inherited. Deployments configure these mechanisms per their requirements. The formalization here identifies the mechanisms as necessary architectural commitments, not as specifications of implementation technology.

Level-distinguishability is **not** a single mechanism. The six mechanisms identified are each necessary; none is individually sufficient. A deployment that implements type declarations but not scope-validation rules has partial distinguishability that will erode under operational pressure. A deployment that implements cross-level operation provenance but not level-specific testing has a record of cross-level events but no verification that level-distinct properties hold.

Level-distinguishability does **not** replace artifact-level governance. Governance properties propagate per B2.08's relational role-membership structure; distinguishability mechanisms add level-aware semantics to operations without superseding the artifact-level governance commitments inherited from Paper 1. Level-distinguishability and artifact-level governance are complementary, not alternatives.

Level-distinguishability is **not** verified by a single operational test. The operational test per §8 below identifies the architectural mechanisms as the test target; verifying that level-distinguishability is maintained requires that multiple test criteria be satisfied—type declarations present and complete, scope-validation rules operational, cross-level provenance complete, level-specific governance records accurate, level-specific tests run and passed, inspection semantics level-aware. The multi-criteria character of the test reflects the multi-mechanism character of distinguishability itself.

8. Operational Test

Test: Given a CKS deployment with multiple artifacts across the three levels, does every artifact carry a substrate-resident type declaration with complete provenance per A2.40; are scope-validation rules per A2.04 operational and enforcing level boundaries for level-confined operations; are cross-level operations—where they occur per B1.19 or B1.14—recorded with complete cross-level provenance and authorized by explicit scope-crossing rules; does level-specific governance per A2.47 assign authority at the declared level-scope of each governance actor; do level-specific tests per A5.01–A5.16 run at each level and verify level-distinct properties; and do inspection semantics per A2.01 return level-distinguished content when queried at a given level?

If all six criteria are met, level-distinguishability operational mechanisms are deployed and level-distinct scope per B2.07 is operationally maintained. If any criterion fails, level-distinguishability is partial and level confusion is operationally possible.

9. Position in the B1.02 Decomposition and Phase B2 Progression

Naming level-distinguishability operational mechanisms as standalone formalization matters because the mechanisms are the operational foundation that makes the rest of the three-level structure architecturally stable. B2.07's level-distinct scope property is a commitment about what the levels are. B2.08's level-membership as substrate-resident is a commitment about how membership is recorded. Without B2.09's operational mechanisms, both of those commitments would be architectural intentions that erode through deployment evolution. With B2.09's mechanisms, both commitments are enforced operationally: level-distinct scope per B2.07 is maintained because scope-validation rules prevent inadvertent scope crossing; level-membership per B2.08 is maintained because type declarations give each artifact a substrate-resident level-type that is governed and provenance-tracked.

The three-note sequence—B2.07, B2.08, B2.09—constitutes a complete decomposition of the structural properties of B1.02 short of instantiation: what the levels are, how membership is recorded,

and how the levels are kept distinct operationally. B2.10 will close the decomposition by formalizing level-instantiation patterns—the specific forms that cell, aspect, and Self instantiation take at deployment. Together, B2.07 through B2.10 provide the operational-variant decomposition of B1.02 sufficient to cover the main patentable territory in three-level structure: the scope property, the membership mechanism, the distinguishability mechanisms, and the instantiation patterns.

After B2.10, Phase B2 moves to the B1.03 cell-level decomposition, beginning with notes B2.11–B2.14, which treat cell scope, aspect scope, Self scope, and relational role membership as standalone derivations. The decomposition continues through the remaining Phase B2 notes across the full set of B1-series commitments.

Source Paper

Li, Wenxin. “The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance.” April 2026. Core theory section: “Three levels of structure,” §5.2; “Level relationships,” §5.2.

Prior Notes in This Series

Li, Wenxin. “Three Architectural Levels: Cell, Aspect, Self with Relational Role Membership” (B1.02). April 2026.

Li, Wenxin. “Level-Distinct Scope Property — Decomposing B1.02” (B2.07). May 2026.

Li, Wenxin. “Level-Membership as Substrate-Resident — Decomposing B1.02” (B2.08). May 2026.

Foundational Paper

Li, Wenxin. “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.” April 2026.